Living Earth Cube-in-a-Box - Admin Guide

Bruno Chatenoux

2026-03-05

Table of contents

1 Overview

Welcome to the Living Earth Cube-in-a-Box Admin Manual. This guide is intended for system administrators responsible for setting up, configuring, and maintaining the Open Data Cube (ODC) environment.

The Living Earth Cube-in-a-Box is a streamlined platform for running the Open Data Cube. It leverages Docker and GNU Make to simplify the deployment of a full stack including:

- **JupyterHub**: Multi-user server management that provides each user with a **JupyterLab** notebook environment.
- **PostgreSQL/PostGIS**: Database for ODC metadata.
- **Explorer**: Web UI for browsing datasets.
- **Traefik**: Reverse proxy and edge router.

The current repository is a fork of https://github.com/opendatacube/cube-in-a-box with the following major changes:

- Multi-user JupyterHub
- shared folders for collaboration
- Planetary Computer as a STAC datastore
- ODC Explorer
- Traefik integration as a reverse proxy
- Multi-architecture support (AMD64 & ARM64)
- Dask integration for parallel processing
- Admin and User documentation (Quarto)

All the developments have made possible thanks to the financial support of the European Union 'Horizon Europe Program' that funded the LandShift[1] (Grant Agreement no. 101182007), Nostradamus[2] (Grant Agreement no. 101134888), and NEMESIS[3] (Grant Agreement no. 101219087) projects.

[1] https://landshift.eu/
[2] https://nostradamus-project.eu
[3] https://www.nemesis-soil.eu/

2 Prerequisites

This project is run via `docker compose` and a `Makefile`. Ensure your system meets the following requirements before starting:

2.1 Linux

- **Docker Engine**: Official Instructions[1].
- **Docker Compose**: Usually included with Docker Engine.
- **GNU Make**: Install via your package manager (e.g. `sudo apt update && sudo apt install make`).

2.2 macOS

- **Docker Desktop**: Official Instructions[2].
- **Make**: Available via Xcode Command Line Tools (`xcode-select --install`).

2.3 Windows (WSL2 Recommended)

1. **WSL2**: Install WSL2[3] and a Linux distribution (e.g., Ubuntu).
2. **Docker Desktop**: Install Docker Desktop for Windows[4]. In settings, enable **Use WSL2 based engine** and **WSL Integration** for your Linux distribution.
3. **Make**: Open your WSL terminal and install Make (`sudo apt update && sudo apt install -y make`).

> Notes
>
> - Run all `make ...` commands **from the WSL terminal** (not PowerShell) to ensure a consistent Linux-like environment.
> - Instead of keeping this project in your regular Windows folders (like your `C:` drive, which appears as `/mnt/c/...` in Linux), save it directly inside your Linux environment (for example, in a `~/projects/` folder). This will make the system run much faster.

[1] https://docs.docker.com/engine/install/
[2] https://docs.docker.com/desktop/setup/install/mac-install/
[3] https://learn.microsoft.com/en-us/windows/wsl/install
[4] https://docs.docker.com/desktop/setup/install/windows-install/

2.4 Quick verification

Once installed, you should be able to run:

```
make --version
docker --version
docker compose version
```

3 Repository Structure

Core Files:

- `Makefile`: Main entry point for commands (setup, indexing, maintenance).
- `docker-compose.yml` / `docker-compose.dev.yml`: Service definitions for prod and dev environments.
- `.env.default` / `.env`: Environment variables for system configuration.
- `Dockerfile` / `docker-bake.hcl`: Definitions for building the Docker images.
- `index-from-config.sh` / `product-from-config.sh`: Config-driven STAC indexing and ODC product registration.
- `config/indexation/`: YAML preset for `INDEX_CONFIG` (`default.yaml`).
- `requirements.txt`: Python package dependencies for the Jupyter environment.
- `distributed.yaml`: Configuration for distributed Dask processing.

Key Directories:

- `data/`: Database storage, local files, and user workspaces.
- `datacube-explorer/`: Web UI configuration and customization.
- `hub/`: JupyterHub setup, authentication, and Docker spawner logic.
- `config/indexation/`: STAC indexation preset (`default.yaml`, `INDEX_CONFIG`).
- `config/products/`: ODC product YAMLs by source (`pc/`, `cop/`).
- `quarto/`: Documentation source (including this manual).
- `scripts/`: Data processing and system maintenance utilities.
- `shared/`: Read-only templates and shared data for all JupyterHub users.

4 First-Time Setup

> **!** Execution Context
>
> All admin operations in this chapter must be executed from the **project root directory** (where the `Makefile` and `docker-compose.yml` reside).

This repository uses environment variables to configure the local domain, database credentials, and the Jupyter password.

1. Create a `.env` file (Docker Compose reads `.env` by default):

```
cp .env.default .env
```

2. Edit `.env` to match your setup:

   - Set `DOMAIN` to your local domain (e.g. `localhost` or a IP)
   - Set strong passwords for `POSTGRES_PASS`
   - Configure `JUPYTERHUB_ADMINS` **(with at least one admin username)**
   - Optionally add regular users to `JUPYTERHUB_USERS`

   Required variables:

Table 4.1: Required environment variables

Variable	Required	Example	Description
`DOMAIN`	Yes	`localhost`	Hostname used to access the web endpoints (Jupyter/Explorer).
`IMAGE_VERSION`	Yes	`20260211`	Version of the images to use.
`POSTGRES_HOSTNAME`	Yes	`postgres`	Hostname used to access the PostgreSQL database.
`POSTGRES_PORT`	Yes	`5432`	Port used to access the PostgreSQL database.
`POSTGRES_DBNAME`	Yes	`opendatacube`	PostgreSQL database name used by Open Data Cube.
`POSTGRES_USER`	Yes	`opendatacube`	PostgreSQL user for the Open Data Cube database.
`POSTGRES_PASS`	Yes	`a-strong-password`	PostgreSQL password for the Open Data Cube database.
`JUPYTERHUB_ADMINS`	Yes	`admin,bruno`	Comma-separated list of JupyterHub admin usernames.
`JUPYTERHUB_USERS`	No	`guest,alice,bob`	Comma-separated list of authorized non-admin usernames.
`CDSE_S3_ACCESS_KEY`	No	*(empty)*	Copernicus Data Space S3 access key (required for CDSE-indexed products).
`CDSE_S3_SECRET_KEY`	No	*(empty)*	Copernicus Data Space S3 secret key.
`CDSE_S3_ENDPOINT`	No	`https://eodata.dataspace.copernicus.eu`	CDSE S3 API endpoint.

3. First-time setup

- Using default parameters:

```
make setup
```

- With a specific area/time (`BBOX`, `DATETIME`):

```
# Switzerland 1 year
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31

# Switzerland all years (till end 2025, might take a while)
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=1984-01-01/2025-12-31
```

Table 4.2: `make setup` parameters

Parameter	Format	Description
`BBOX`	`lon_min,lat_min,lon_max,lat_max`	Spatial extent in decimal degrees. Use a tool like bboxfinder.com[1] (copy the **Box** values).
`DATETIME`	`start/end`	Temporal extent. Use `YYYY-MM-DD` format (e.g., `2020-01-01/2020-12-31`).
`INDEX_CONFIG`	path to YAML	STAC indexation preset (default: `config/indexation/default.yaml`).

! Copernicus Data Space products

`config/indexation/default.yaml` indexes Sentinel-2 (`s2_l2a_cdse`) and CLCplus from Copernicus Data Space STAC[2], plus Planetary Computer products (including `s2_l2a_pc`).

- Set `CDSE_S3_ACCESS_KEY` and `CDSE_S3_SECRET_KEY` in `.env` (S3 credentials[3]). These keys are required for **indexing**, **Explorer downloads**, and **notebook access** to CDSE products. Explorer download links are signed automatically (no proxy secret to configure); notebooks read CDSE data directly via `s3://` and do not use that signing path.
- **S3 endpoint pinning:** when `CDSE_S3_ACCESS_KEY`/`CDSE_S3_SECRET_KEY` are set, notebook and Explorer S3 access is pinned to the CDSE endpoint (`AWS_S3_ENDPOINT=eodata...`, `AWS_VIRTUAL_HOSTING=FALSE`). Generic `s3://` reads from other providers (e.g. AWS-hosted buckets) will not work in that configuration; Planetary Computer assets are unaffected (served over HTTPS with SAS tokens).
- CLCplus is **Europe-only** — use a European `BBOX` (e.g. Switzerland below).
- If CDSE indexing fails with transient errors (e.g. 504 or STAC API timeouts), retry with `make index-serie` (`PARALLELISM=1`) for a gentler load on the catalog. This is a mitigation for server flakiness, not evidence of access limits.
- In notebooks, use `get_patch_url` from `utils.le_dc` (CDSE access uses `s3://` paths with GDAL header auth via `utils/le_cdse_s3.py`; presigned URLs are not supported by CDSE):

```
from utils.le_dc import get_patch_url

ds = dc.load(..., patch_url=get_patch_url(dc, product))
```

  Copying `notebooks_demo/` to another location includes `utils/le_cdse_s3.py`; no separate signing module is required. Ensure `CDSE_S3_ACCESS_KEY` and `CDSE_S3_SECRET_KEY` are set, then restart your Jupyter server so Dask workers inherit the credentials.

[1] http://bboxfinder.com

[3] https://stac.dataspace.copernicus.eu/v1/

- European setup example (CLCplus):

```
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31
```

> **Note**
>
> `make product` and `make index` both use `INDEX_CONFIG` (default: `config/indexation/default.yaml`). Only products referenced via `product_file` in that config are registered and indexed. Sentinel-2 L2A is split into `s2_l2a_pc` (Planetary Computer) and `s2_l2a_cdse` (CDSE); other ODC names are unchanged (`nasadem`, etc.). Source is recorded in each product YAML under `metadata.product.source`.

https://documentation.dataspace.copernicus.eu/APIs/S3.html

5 User Management

JupyterHub uses NativeAuthenticator with a custom signup handler that restricts access to pre-authorized users only.

But admin users can add new users through the JupyterHub admin panel.

5.1 How User Authorization Works

1. **Authorized Users**: Only users listed in `JUPYTERHUB_ADMINS` or `JUPYTERHUB_USERS` in the `.env` file (or manually added by an admin user) can successfully sign up.
2. **Unauthorized Users**: Unauthorized users will see an error message directing them to contact the administrator if they try to sign up.
3. **Self-Service Signup**: Authorized users can create their own accounts via the signup page.
4. **Admin Creation**: Administrators can also create user accounts through the JupyterHub admin panel.

5.2 Adding Authorized Users

Method 1: Via `.env` file (Recommended for initial setup)

Edit the `.env` file:

```
# Admin users (have full control over JupyterHub)
JUPYTERHUB_ADMINS=admin,bob

# Regular users (can only access their own notebooks)
JUPYTERHUB_USERS=guest,charlie
```

> ⚠ Notes
>
> - You need to have at least one admin user (`JUPYTERHUB_ADMINS`)!
> - If you add a user this way while the system is running, you need to restart JupyterHub to apply changes:
>
>   ```
>   docker-compose restart jupyterhub
>   ```

Users can now visit `http://<DOMAIN>/jupyter/hub/signup` to create their accounts.

Method 2: Via JupyterHub Admin Panel (Once the system is running, for ad-hoc user additions)

1. Log in as an admin user.

2. File > Hub Control Panel > Admin, or navigate to http://<DOMAIN>/jupyter/hub/admin.

3. Click Add Users.
4. Enter the username and click Add.
5. The user is created immediately and can sign up at http://<DOMAIN>/jupyter/hub/signup.

5.3 User Signup Flow

For Authorized Users:

1. Visit http://<DOMAIN>/jupyter/hub/signup.

2. Fill in username (must match an authorized one) and password.
3. Submit the form.
4. See success message: "The signup was successful! You can now go to the home page and log in to the system."
5. Log in at http://<DOMAIN>/jupyter/hub/login.

For Unauthorized Users:

1. Contact the administrator to be added.

5.4 Managing Existing Users

View all users:

1. Log in as admin.

2. Navigate to http://<DOMAIN>/jupyter/hub/admin.
3. You'll see a list of all users with their status and last activity.

Edit user:

1. Click "Stop Server" and then "Edit User" next to any user.

2. You can make them admin, delete them, or manage their servers.

Delete user:

1. Click "Edit User" → "Delete User".
2. This removes the user account but doesn't delete their notebook files (stored in jupyterhub-user-<username> volume).
3. Execute make purge-user HUB_USER=<username> CONFIRM=1 to delete the user's volume.
4. Remove user from .env file if required.

Password reset or recovery:

1. Using the JupyterHub Admin interface, delete the user and re-create them, **without deleting their user volume**.
2. Inform the user they will have to sign up again.

5.5 User Data and Notebooks

Each user's notebooks are stored in a Docker volume named `jupyterhub-user-<username>`. These volumes persist even if the user account is deleted.

Backup user notebooks:

```
docker run --rm -v jupyterhub-user-<username>:/source -v $(pwd)/backups:/backup
 ↪  alpine tar czf /backup/user-<username>-notebooks.tar.gz -C /source .
```

Restore user notebooks:

```
docker run --rm -v jupyterhub-user-<username>:/target -v $(pwd)/backups:/backup
 ↪  alpine tar xzf /backup/user-<username>-notebooks.tar.gz -C /target
```

Remove all user volumes (⚠️⚠️ use with extra care):

```
make purge-users CONFIRM=1  # ⚠️⚠️
```

Remove a specific user volume (⚠️ use with care):

```
make purge-user HUB_USER=alice CONFIRM=1  # ⚠️
```

6 CiaB Management with make

The system uses GNU Make[1] as a task automation tool to wrap complex Docker and data indexing workflows into simple commands. Instead of manually executing long command-line sequences, you use make targets to trigger predefined administrative tasks.

To see the full list of available targets on your machine, run:

```
make help
```

6.1 Target reference (from make help)

Table 6.1: make target reference

Command	Description
Runtime Control	
make up	Start the environment in the background (then open Jupyter in your browser)
make down	Stop the running services (keeps your data and images)
make status	Show what is running (containers and their status)
make logs	Show live logs from all services (useful for troubleshooting)
make shell	Open a terminal inside the Jupyter container (requires HUB_USER)
make wait-for-db	Wait for PostgreSQL to be ready to accept connections
Setup & Init	
make setup	First-time setup (mode-dependent: uses pull in prod, build in dev)
make init	Initialize the Open Data Cube database (run once after setup)
make build	Build the images locally
make build-nocache	Build the images locally from scratch
make pull	Download all service images (recommended before first run in prod mode)
Data & Indexing	
make product	Load ODC product definitions from INDEX_CONFIG into the database
make index	Index products from INDEX_CONFIG (parallel; uses BBOX, DATETIME)
make index-parallel	Same as make index
make index-serie	Index products from INDEX_CONFIG sequentially (PARALLELISM=1)
make update-explorer	Rebuild the Explorer index so datasets appear in the web UI
Maintenance	
make backup	Create a backup of the PostgreSQL database
make restore	Restore PostgreSQL database from a backup file (requires BACKUP_FILE and CONFIRM=1)
make clean	Stop everything and remove containers, volumes, and built images
make purge-data	Delete local data in ./data (pg, local_data, shared). Irreversible; requires CONFIRM=1
make purge-user	Remove a specific user container and volume. Irreversible; requires HUB_USER and CONFIRM=1
make purge-users	Remove all spawned JupyterHub user containers and volumes. Irreversible; requires CONFIRM=1
Advanced	
make release-push	Build and push multi-architecture production images to the configured container registry
make help	Show available commands

6.2 Common usage patterns

- First-time setup (default parameters):

[1] https://www.gnu.org/software/make/

```
make setup
```

- Setup with a specific area/time (`BBOX` and `DATETIME`):

```
# Switzerland 1 year
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31

# Switzerland all years (till end 2025, might take a while)
make setup BBOX=5.95,45.81,10.50,47.81 DATETIME=1984-01-01/2025-12-31
```

- Start/stop and troubleshoot:

```
make up
make status
make logs
make down
```

- Reset options (⚠️ use with care):

```
# Stop everything and remove containers/volumes/images
make clean

# ⚠️ Irreversible: delete local data in ./data (requires confirmation)
make purge-data CONFIRM=1

# ⚠️⚠️ TOTAL WIPE OUT (USE WITH EXTRA CARE !!!)
make clean && make purge-data CONFIRM=1 && make purge-users CONFIRM=1
# then check for any remaining resources
docker ps -a && docker images -a && docker volume ls && ls -la ./data
```

6.3 STAC indexation configs

`make product` and `make index` both read the same `INDEX_CONFIG` YAML. Only products listed in that file are registered and indexed.

Indexation presets live in `config/indexation/`. ODC product definitions live in `config/products/pc/` (Planetary Computer) and `config/products/cop/` (Copernicus CDSE), with `metadata.product.source` set in each YAML.

Config	Source(s)	Products
`default.yaml` (default)	Planetary Computer + CDSE	11 products: `s2_l2a_pc`, `s2_l2a_cdse`, CLCplus (CDSE); S1 RTC, Landsat, NASADEM, ESA WorldCover, ESRI land cover (PC)

```
# Default (all products)
make index BBOX=25,20,35,30 DATETIME=2021-12-01/2021-12-31

# Per-product timing report at the end
make index TIME_INDEX=1 BBOX=25,20,35,30 DATETIME=2021-12-01/2021-12-31

# European BBOX recommended for CLCplus
make index BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31
```

```
# Sequential indexing (gentler on flaky catalogs such as CDSE)
make index-serie BBOX=5.95,45.81,10.50,47.81 DATETIME=2024-01-01/2024-12-31
```

If CDSE indexing fails with transient errors (e.g. 504 or STAC API timeouts), retry with `make index-serie` (`PARALLELISM=1`). This reduces concurrent load on the catalog; it is a mitigation for server flakiness, not evidence of access limits.

7 Execution Modes

Living Earth Cube-in-a-Box supports two execution modes: **Production** (`prod`) and **Development** (`dev`). The `MODE` environment variable determines which Docker Compose configuration is used.

Table 7.1: Execution modes comparison

Mode	Purpose	Image Source	Compose Files
Production (`prod`)	Standard deployment (default).	Pre-built remote images.	`docker-compose.yml`
Development (`dev`)	Local testing and modifications.	Locally built images (`:dev`).	`+ docker-compose.dev.yml`

7.1 Production Mode (`prod`)

This is the default mode. It pulls official images from the registry and is optimized for stability.

```
# Explicitly set (optional)
export MODE=prod
make up
```

7.2 Development Mode (`dev`)

Use this mode to modify Dockerfiles or core configurations. In this mode, `make setup` builds images locally instead of pulling them.

Method 1: Session-wide (Recommended)

This ensures all subsequent `make` commands (up, status, logs, etc.) use the correct mode.

```
export MODE=dev
make setup
make up
```

Method 2: One-off Command

You can pass the variable directly, but you must remember to include it in **every** command.

```
make up MODE=dev
make status MODE=dev
```

Switching Back

To return to production mode:

```
unset MODE
# or
export MODE=prod
```

7.3 Verification

Run `make help` to verify the active mode:

```
Current mode: dev
...
```

8 Shared Directory

This directory is shared among users of the JupyterHub instance.

The primary purpose of this directory is to facilitate file sharing and collaboration between users.

The `./shared/` directory contains:

- **Static Content**: Directories and files directly under the root are read-only for everyone in the Jupyter interface.
- **User Directories**: Directories and files under `./shared/all_users/` are visible to all users in the Jupyter interface as read-only, with only their own directory as read-write. This allows users to copy notebooks from others but not modify their work directly.

> **Note**
>
> CiaB Admin can add/remove/modify `./shared/` directory content at any time on the host machine. The changes will be visible to all users in the Jupyter interface immediately.

9 Backup and Restore

The `./data/` subdirectories contain persistent data and should be backed up regularly:

- `./data/pg/` - PostgreSQL database files
- `./data/local_data/` - Local data cache
- `./data/jupyterhub_data/` - JupyterHub configuration and user data

User notebooks are stored in Docker volumes named `jupyterhub-user-<username>`

This chapter describes how to backup and restore the CiaB Database and user volumes to and from the `./backups/` directory.

9.1 Database backup procedure

Database backup:

To create a backup of your PostgreSQL database:

```
make backup
```

This will create a timestamped SQL dump file in the `./backups` directory.
Example: `./backups/opendatacube_20260121_141530.sql`.

Restore Database:

To restore a database from a backup file:

```
make restore BACKUP_FILE=./backups/opendatacube_20260121_141530.sql CONFIRM=1
```

> ⚠ Notes
>
> Restoring will overwrite your current database. Make sure you have a recent backup before proceeding.

9.2 Volume backup procedure

Manual volume backup:

```
# Backup user notebooks
docker run --rm -v jupyterhub-user-<username>:/source -v $(pwd)/backups:/backup
 ↪  alpine tar czf /backup/user-<username>-notebooks.tar.gz -C /source .

# Backup all data directories
tar czf backups/data-backup-$(date +%Y%m%d).tar.gz ./data/
```

Restore user notebooks:

```
# Restore user notebooks
docker run --rm -v jupyterhub-user-<username>:/target -v $(pwd)/backups:/backup
 ↪  alpine tar xzf /backup/user-<username>-notebooks.tar.gz -C /target
```

10 Security Best Practices

1. **Use strong passwords** for admin accounts
2. **Regularly review** the user list in the admin panel
3. **Remove unused accounts** to minimize security risks
4. **Backup user data** regularly (see Backup and Restore section)
5. **Keep admin users minimal** - only trusted users should have admin access

11 License

This project is a fork of https://github.com/opendatacube/cube-in-a-box, and consequently inherits its **MIT License**.

Figure 11.1: License: MIT

[1] https://opensource.org/license/mit